Parte 3 – Código de mensajería

Video Pedidos360 · Evaluación 2 (DSY1107) · Integrante 3 · cierra el video

Duración	2:00 – 2:30 minutos
Qué se ve en pantalla	Bloque 1: el diagrama de la última página. Bloques 2 a 6: el fragmento de código de cada bloque (son copias del código real, recortadas). Haz zoom para que se lea.
Cómo grabar	Win + Alt + R (grabador de Windows), OBS, o una reunión de Teams/Zoom grabada compartiendo pantalla. 1080p, con voz, sin música.
Entrega	Archivo `parte3-integrante3.mp4` enviado a Sebastián.

No necesitas AWS ni el proyecto instalado. Si Sebastián sube el repositorio a GitHub, puedes mostrar ahí los mismos archivos; el texto no cambia. Si te equivocas, pausa dos segundos y repite desde el inicio de ese bloque.

Guion

1. Estructura · 15 s

Muestra: el diagrama (última página). Señala backend, procesador-service y rabbit-admin-service.

> "El backend son tres microservicios Spring Boot: `backend`, que guarda el pedido y publica los mensajes; `procesador-service`, con los consumidores; y `rabbit-admin-service`, que administra RabbitMQ por API REST."

2. Nombres centralizados · 25 s

Muestra: `procesador-service/src/main/resources/application.yml`

```
pedidos360:
  messaging:
    exchanges:
      events: pedidos.events.exchange        # TOPIC
      tasks: pedidos.tasks.exchange          # DIRECT
      dead-letter: pedidos.dlx               # DIRECT - DLX común
    queues:
      tickets:
        name: tickets.queue
        binding-key: pedido.creado
        dlq: tickets.dlq
        dead-letter-key: tickets.dead
    dead-letter:                             # retención de las DLQ
      retention-ms: 86400000                 # 24 h
      max-length: 10000
    retry:
      max-attempts: 3
      initial-interval-ms: 1000
      multiplier: 2.0
```

> "Todos los nombres de colas, exchanges y bindings viven en el application.yml de cada servicio y se leen con `@ConfigurationProperties`. Ninguna clase escribe un nombre a mano."

3. Beans por caso de uso · 25 s

Muestra: `config/TicketsTopologyConfig.java` (hay una igual para notificaciones y otra para documentos).

```
@Configuration
public class TicketsTopologyConfig {
    // cola principal, con x-dead-letter-exchange
    @Bean Queue ticketsQueue(MessagingProperties props) {
        return TopologySupport.mainQueue(props.queues().tickets(), props);
    }
    // DLQ, con TTL y max-length (retención)
    @Bean Queue ticketsDlq(MessagingProperties props) {
        return TopologySupport.deadLetterQueue(props.queues().tickets(), props);
    }
    @Bean Binding ticketsBinding(@Qualifier("ticketsQueue") Queue queue,
            TopicExchange eventsExchange, MessagingProperties props) {
        return BindingBuilder.bind(queue).to(eventsExchange)
                .with(props.queues().tickets().bindingKey());
    }
    @Bean Binding ticketsDlqBinding(@Qualifier("ticketsDlq") Queue dlq,
            @Qualifier("deadLetterExchange") DirectExchange dlx,
            MessagingProperties props) {
        return BindingBuilder.bind(dlq).to(dlx)
                .with(props.queues().tickets().deadLetterKey());
    }
}
```

> "La configuración de mensajería está aislada en el paquete config: un archivo por caso de uso con su cola, su DLQ y sus bindings."

4. Consumidores por dominio · 20 s

Muestra: `tickets/TicketConsumer.java`

```
@RabbitListener(queues = "${pedidos360.messaging.queues.tickets.name}")
public void onMessage(Message message, Channel channel) {
    processor.process(NAME, message, channel,
            PedidoCreadoEvent.class, service::generar);
}
```

> "Hay un paquete por dominio: notificaciones, tickets y documentos. El consumidor solo recibe el mensaje; la lógica de negocio está en `TicketService`, que no conoce RabbitMQ."

5. ACK y manejo de errores · 40 s · el bloque más importante de la pauta

Muestra: `messaging/AckingMessageProcessor.java`, con `acknowledge-mode: manual` y `prefetch: 1`.

```
while (true) {
    attempt++;
    try {
        handler.accept(payload);
        channel.basicAck(tag, false);              // éxito
        return;
    } catch (PermanentProcessingException ex) {
        channel.basicNack(tag, false, false);      // permanente -> DLQ
        return;
    } catch (RuntimeException ex) {                // recuperable
        if (attempt >= retry.maxAttempts()) {
            channel.basicNack(tag, false, false);  // agotado -> DLQ
            return;
        }
        sleeper.sleep(delay);                      // espera 1 s, 2 s
        delay = Math.min((long) (delay * retry.multiplier()),
                retry.maxIntervalMs());
    }
}
```

> "Cada consumidor confirma de forma explícita. Si el procesamiento sale bien hace ACK. Si el error es permanente, como un JSON inválido, hace NACK sin reencolar y el mensaje va directo a la DLQ. Si es recuperable, reintenta con espera creciente y, al agotar los tres intentos, lo manda a la DLQ. Nunca reencolamos en bucle, y un mensaje duplicado se confirma sin reprocesarlo."

6. Microservicio administrador · 30 s

Endpoints: `POST /api/queues`, `GET` y `DELETE /api/queues/{name}`, `POST` y `DELETE /api/exchanges`, `POST` y `DELETE /api/bindings`, `GET /api/monitoring/queues`. Muestra: `dto/QueueCreateRequest.java`

```
public record QueueCreateRequest(
        @NotBlank(message = "name es obligatorio")
        @Pattern(regexp = ValidationPatterns.BROKER_NAME,
                 message = ValidationPatterns.BROKER_NAME_MESSAGE)
        String name,
        Boolean durable,
        Boolean autoDelete,
        String deadLetterExchange,
        String deadLetterRoutingKey,
        @PositiveOrZero(message = "messageTtlMs no puede ser negativo")
        Long messageTtlMs,
        @Positive(message = "maxLength debe ser mayor que 0")
        Long maxLength) { }
```

> "Toda la lógica de administración está en `RabbitAdminService`; los controladores solo llaman a sus métodos. Las entradas se validan con DTO: no se puede crear una cola con nombre vacío, con caracteres inválidos ni con el prefijo reservado `amq.`. Los errores vuelven en JSON con 400, 404 o 409, y modificar RabbitMQ exige el rol Admin."

7. Pruebas · 10 s

> "El proyecto tiene 36 pruebas automáticas: el flujo de ACK, NACK y reintentos, las validaciones y la seguridad del administrador, y el publicador del backend."

8. Cierre · 15 s

> "En resumen, Pedidos360 ahora procesa notificaciones, tickets y documentos de forma asíncrona con RabbitMQ en un clúster de dos nodos, con reintentos, DLQ y monitoreo, sin afectar la lógica que ya existía. Muchas gracias."

Antes de enviar

☐ Dura entre 2:00 y 2:30.
☐ La voz se entiende de principio a fin, sin ruido de fondo.
☐ El código se lee en el video (haz zoom si hace falta).
☐ No aparecen chats, correos ni otras pestañas.

☐ Termina con "Muchas gracias."
☐ El archivo se llama `parte3-integrante3.mp4` y ya se lo enviaste a Sebastián.

Si el docente te pregunta

¿Por qué ACK manual?	Para que el consumidor decida el destino de cada mensaje: ACK si salió bien, reintento si es transitorio, NACK sin reencolar hacia la DLQ si no tiene arreglo.
¿Cómo evitan un bucle infinito?	Máximo 3 intentos con espera de 1 s y 2 s; después NACK con `requeue=false` y el mensaje sale de la cola principal.
¿Diferencia entre DLX y DLQ?	El DLX es un exchange que enruta los mensajes rechazados; la DLQ es la cola donde quedan guardados (24 h, máximo 10.000).
¿Para qué `@ConfigurationProperties`?	Lee los nombres del application.yml en un objeto tipado, así ningún nombre queda escrito a mano en el código.
¿Y si un mensaje llega dos veces?	El procesador recuerda el `messageId` de los ya procesados y confirma el duplicado sin repetir el trabajo.
¿Y si RabbitMQ se cae?	El pedido se guarda igual en la base de datos; el error queda en el log y no afecta la respuesta al usuario.

Un pedido dispara tres tareas en RabbitMQ; lo que falla queda en su DLQ

Pedidos360 en AWS · en color, la ruta de excepción (DLX y DLQ)

